

Osnove programiranja 2

Projektni zadatak

Sistem za navigaciju korištenjem grafova

1. Uvod

Ovaj projektni zadatak bavi se implementacijom sistema za navigaciju koji koristi grafovske strukture podataka za modelovanje mapa gradova. Graf je matematička struktura koja se sastoji od skupa cvorova i grana koje ih međusobno povezuju. U kontekstu ovog projekta, cvorovi predstavljaju lokacije unutar gradova (kafici, restorani, pabovi), a grane sa svojim tezinama predstavljaju puteve između tih lokacija i njihovu međusobnu udaljenost.

Rezultat projekta je konzolna aplikacija koja korisniku nudi mogućnost da izabere jednu od devet ponudjenih mapa gradova. Korisnik unosi početnu i krajnju lokaciju, a program pronalazi i prikazuje najkraci put između njih. Takođe, korisnik može unijeti dodatne obavezne medjustanice kroz koje put mora prolaziti. Ukoliko traženi put ne postoji, ispisuje se odgovarajuća poruka.

Dokumentacija je organizovana na sljedeći način: poglavlje 2 opisuje strukture podataka, poglavlje 3 detaljno opisuje primijenjene algoritme, poglavlje 4 prikazuje i analizira rezultate mjerenja, a poglavlje 5 donosi zaključke.

2. Strukture podataka

Za implementaciju grafova odabrana je reprezentacija pomocu liste susjedstva zasnovane na jednostruko povezanoj listi (JPL). Ova reprezentacija je efikasnija od matrice susjedstva za rijetke grafove jer zauzima memoriju proporcionalno broju grana. Koriste se tri medjusobno povezane strukture podataka.

2.1 Cvor liste susjedstva

Svaki element liste susjedstva predstavlja jednu granu grafa. Sadrzi indeks susjednog cvora, tezinu grane i pokazivac na sljedeci element liste:

```
typedef struct cvor cvor;
struct cvor {
    int    susjed;    /* indeks susjednog cvora */
    int    tezina;    /* tezina grane (1-50) */
    cvor* sljedeci;   /* sljedeci element JPL */
};
```

2.2 Lista susjedstva

Lista susjedstva svakog cvora modelovana je strukturom sa pokazivcem na prvi element:

```
typedef struct {
    cvor* prvi;
} lista_susjeda;
```

2.3 Graf

Centralna struktura projekta sadrzi sve potrebne informacije o mapi: broj cvorova, gustinu, naziv, dinamicki niz imena lokacija i dinamicki niz listi susjedstva:

```
typedef struct {
    int        broj_cvorova;
    float      gustina;
    char       naziv[60];
    char       prefiks[20];
    char**     ime_cvora;
    lista_susjeda* susjedi;
} graf;
```

Svi grafovi alociraju se na heap memoriji funkcijom malloc, a u funkciji main cuva se niz pokazivaca graf* mape[9]. Ovako se izbjegava prekoracenje steka (stack overflow) koje bi nastalo alociranjem velikih struktura kao lokalnih varijabli.

3. Algoritmi

3.1 Algoritam za kreiranje grafova

Kreirano je devet grafova variranjem broja cvorova (tri opsega) i gustine (tri vrijednosti: 0.3, 0.6 i 0.8). Svaka kombinacija daje jedan od devet grafova:

Graf	Naziv mape	Cvorovi	Gustina	Grupa
1	Pabovi u Beogradu	800	0.3	I
2	Kafici u Banjoj Luci	500	0.6	I
3	Restorani u Sarajevu	120	0.8	I
4	Pabovi u Zagrebu	2800	0.3	II
5	Restorani u Splitu	3800	0.6	II
6	Kafici u Splitu	4500	0.8	II
7	Kafici u Skoplju	9400	0.3	III
8	Restorani u Becu	6500	0.6	III
9	Pabovi u Podgorici	8300	0.8	III

Tabela 1: Pregled kreiranih grafova

Svaki graf se kreira u dva koraka. Prvo se formira lanac 1 - 2 - 3 - ... - n koji garantuje da je graf barem djelimicno povezan. Zatim se nasumicno dodaju dodatne grane do ciljanog broja određenog gustinom:

$$\text{ciljani_broj_grana} = \text{gustina} \times n \times (n - 1) / 2$$

Tezine grana su prirodni brojevi u opsegu 1 - 50, generisani nasumicno. Grafovi su neusmjereni i ne sadrže petlje - svaka grana se dodaje u oba smjera u listama susjedstva.

3.2 Dijkstrin algoritam

Za pronalazjenje najkraceg puta koristi se Dijkstrin algoritam u osnovnoj $O(N^2)$ implementaciji. Algoritam radi na sljedeći način:

Korak 1 - Inicijalizacija: Distanca do svih cvorova postavlja se na INT_MAX (beskonacno), a distanca do pocetnog cvora na 0. Niz prethodnika inicijalizuje se na -1.

Korak 2 - Odabir minimuma: U svakoj iteraciji bira se neposjeceni cvor u sa najmanjom trenutnom distancom. Ovaj korak se ponavlja N puta.

Korak 3 - Relaksacija: Za cvor u prolazi se kroz listu susjedstva (JPL). Za svakog susjeda v , ako je $udaljenosti[u] + tezina_grane < udaljenosti[v]$, azurira se $udaljenosti[v]$ i $prethodni[v] = u$.

Korak 4 - Rekonstrukcija puta: Put se rekonstruise pracenjem niza prethodnika unazad od cilja do izvora, a zatim se obrce. Ako distanca do cilja ostane INT_MAX - put ne postoji.

3.3 Pronalazenje puta sa medjustanicama

Kada korisnik unese obavezne medjustanice, put se dijeli na segmente: pocetna -> medjustanica 1 -> ... -> krajnja. Za svaki segment pokrece se Dijkstrin algoritam. Program prvo provjerava postoje li svi segmenti, pa tek onda ispisuje cijeli put. Medjustanice su oznacene sa [M], pocetna sa [S], a krajnja sa [C].

4. Rezultati mjerenja vremena izvršavanja

Za svaki graf izvršeno je po pet mjerenja Dijkstrinog algoritma. Prikazana vrijednost je aritmeticka sredina pet mjerenja. Mjerenja su odvojena za dva slucaja: bez medjustanica (jedno pokretanje algoritma) i sa jednom medjustanicom (dva uzastopna pokretanja).

4.1 Bez medjustanica

Graf	Naziv mape	Cvorovi	Gustina	Prosjeak (ms)
1	Pabovi u Beogradu	800	0.3	1.3806
2	Kafici u Banjoj Luci	500	0.6	0.6130
3	Restorani u Sarajevu	120	0.8	0.0598
4	Pabovi u Zagrebu	2800	0.3	13.3194
5	Restorani u Splitu	3800	0.6	23.2528
6	Kafici u Splitu	4500	0.8	4.2590
7	Kafici u Skoplju	9400	0.3	164.8662
8	Restorani u Becu	6500	0.6	65.9200
9	Pabovi u Podgorici	8300	0.8	47.7308

Tabela 2: Prosjeak 5 mjerenja - bez medjustanica

4.2 Sa jednom medjustanicom

Graf	Naziv mape	Cvorovi	Gustina	Prosjeak (ms)
1	Pabovi u Beogradu	800	0.3	2.6664
2	Kafici u Banjoj Luci	500	0.6	1.1152
3	Restorani u Sarajevu	120	0.8	0.1140
4	Pabovi u Zagrebu	2800	0.3	25.9196
5	Restorani u Splitu	3800	0.6	45.3380
6	Kafici u Splitu	4500	0.8	8.3108
7	Kafici u Skoplju	9400	0.3	346.5634
8	Restorani u Becu	6500	0.6	141.8630
9	Pabovi u Podgorici	8300	0.8	95.3918

Tabela 3: Prosjeak 5 mjerenja - sa jednom medjustanicom

4.3 Analiza rezultata

Uticaj broja cvorova: Najznacajniji faktor je broj cvorova. Graf 7 (9400 cvorova) ima vrijeme 164.87 ms, a Graf 3 (120 cvorova) svega 0.06 ms. Ovo potvrđuje kvadratnu složenost $O(N^2)$ - udvostrucavanje broja cvorova približno četverostruko povećava vrijeme izvršavanja.

Uticaj gustine: Veca gustina znaci vise grana pa Dijkstra trosi vise vremena na relaksaciju susjeda u svakoj iteraciji. Ipak, broj cvorova dominantnije utice na ukupno vrijeme.

Uticaj medjustanica: Vrijeme sa medjustanicom je približno dvostruko duze jer se Dijkstrin algoritam pokrce dva puta. Odnos u svim mjerenjima je u opsegu 1.90 - 2.10.

5. Zaključak

U okviru projektnog zadatka uspješno je implementiran sistem za navigaciju zasnovan na grafovskim strukturama. Implementirano je devet grafova u tri grupe po veličini (do 1 000, 1 000 - 5 000 i 5 000 - 10 000 čvorova), svaka sa tri različite gustine (0.3, 0.6, 0.8). Konzolna aplikacija omogućava pronalazanje najkracih puteva sa ili bez obaveznih međjustanica.

Mjerenja su potvrdila kvadratnu složenost Dijkstrinog algoritma. Mali grafovi (do 1 000 čvorova) imaju zanemarljivo kratko vrijeme ispod 2 ms, srednji (1 000 - 5 000) do 24 ms, a veliki (5 000 - 10 000) dostižu i do 165 ms. Svaki dodatni segment puta (međjustanica) približno udvostručuje ukupno vrijeme, jer zahtijeva dodatno pokretanje algoritma.